

Week 2: Intermediate SQL — JOINS and Aggregations

Phase 1: Foundations | Week 2 of 12

Week Overview

Week 2 moves from single-table queries to multi-table analysis. You will master every type of JOIN, understand aggregation with GROUP BY and HAVING, and write your first subqueries. By Friday you can answer virtually any business analytics question involving structured relational data.

Focus: Understand JOIN semantics deeply. Most SQL bugs come from wrong JOIN types.

Learning Objectives

By the end of this week, you will be able to:

- Write INNER, LEFT, RIGHT, FULL OUTER, and CROSS JOINs correctly.
 - Explain when to use each JOIN type with real examples.
 - Use aggregate functions: COUNT, SUM, AVG, MIN, MAX.
 - Write GROUP BY and HAVING clauses correctly.
 - Write correlated and non-correlated subqueries.
 - Use EXISTS and NOT EXISTS for membership tests.
 - Combine result sets with UNION, INTERSECT, EXCEPT.
 - Use DISTINCT and COUNT(DISTINCT col).
-

Required Reading

- `03_Intermediate_SQL/` — All files
-

Daily Schedule

Monday — JOIN Fundamentals (60 min)

Topics:

- The Venn diagram mental model for JOINs
- INNER JOIN — only matching rows
- LEFT OUTER JOIN — all from left, NULLs from right when no match
- RIGHT OUTER JOIN — all from right
- FULL OUTER JOIN — all rows from both sides

```
-- Setup for this week
CREATE TABLE departments (
  dept_id    SERIAL PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  budget     NUMERIC(12,2)
);
```

```

INSERT INTO departments (name, budget) VALUES
  ('Engineering', 500000), ('Marketing', 200000),
  ('HR', 150000), ('Finance', 300000);

-- Add dept_id to employees (from Week 1)
ALTER TABLE employees ADD COLUMN dept_id INTEGER REFERENCES departments(dept_id);
UPDATE employees SET dept_id = 1 WHERE department = 'Engineering';
UPDATE employees SET dept_id = 2 WHERE department = 'Sales';
UPDATE employees SET dept_id = 3 WHERE department = 'HR';

-- INNER JOIN – employees who have a department
SELECT e.first_name, e.last_name, d.name AS dept
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN – all employees, including those without a department
SELECT e.first_name, e.last_name, d.name AS dept
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- RIGHT JOIN – all departments, even empty ones
SELECT e.first_name, d.name AS dept
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- FULL OUTER JOIN
SELECT e.first_name, d.name AS dept
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;

```

Tuesday — Aggregations and GROUP BY (90 min)

Topics:

- COUNT(*) vs. COUNT(col) — the NULL difference
- SUM, AVG, MIN, MAX
- GROUP BY rules — non-aggregate columns must be in GROUP BY
- HAVING — filter on aggregated values (not WHERE)
- ROLLUP and GROUPING SETS (preview)

```

-- Count employees per department
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id
ORDER BY headcount DESC;

-- Average salary per department, only depts with >1 employee
SELECT d.name, AVG(e.salary) AS avg_salary, COUNT(*) AS headcount
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id

```

```

GROUP BY d.name
HAVING COUNT(*) > 1
ORDER BY avg_salary DESC;

-- Department summary with multiple aggregates
SELECT
    d.name                AS department,
    COUNT(e.id)           AS employees,
    SUM(e.salary)         AS total_payroll,
    ROUND(AVG(e.salary), 2) AS avg_salary,
    MIN(e.salary)         AS min_salary,
    MAX(e.salary)         AS max_salary,
    d.budget - SUM(e.salary) AS remaining_budget
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.name, d.budget;

-- ROLLUP for subtotals
SELECT department, COUNT(*), SUM(salary)
FROM employees
GROUP BY ROLLUP (department);

```

Wednesday — Subqueries (90 min)

Topics:

- Scalar subquery (single value)
- Table subquery (inline view)
- Correlated subquery (references outer query)
- IN / NOT IN with subqueries
- EXISTS / NOT EXISTS (preferred over IN for large sets)

```

-- Scalar subquery: employees earning above average
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Table subquery (inline view)
SELECT dept_name, avg_sal
FROM (
    SELECT department AS dept_name, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department
) dept_avgs
WHERE avg_sal > 70000;

-- Correlated subquery: employees earning more than their dept average
SELECT e.first_name, e.salary, e.department
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)

```

```

        FROM employees e2
        WHERE e2.department = e.department
    );

-- EXISTS: departments that have at least one employee
SELECT d.name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
);

-- NOT EXISTS: departments with no employees
SELECT d.name
FROM departments d
WHERE NOT EXISTS (
    SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
);

```

Thursday — Set Operations and DISTINCT (60 min)

Topics:

- UNION vs. UNION ALL (deduplication cost)
- INTERSECT — rows in both
- EXCEPT — rows in first but not second
- DISTINCT vs. GROUP BY for deduplication
- COUNT(DISTINCT col)

```

-- UNION: all employees across two tables (union of names)
SELECT first_name || ' ' || last_name AS name FROM employees
UNION
SELECT name FROM departments; -- mixes types to demonstrate union shape

-- UNION ALL (keeps duplicates, faster)
SELECT department FROM employees WHERE salary > 70000
UNION ALL
SELECT department FROM employees WHERE hire_date > '2023-01-01';

-- INTERSECT: employees in both sets
SELECT id FROM employees WHERE salary > 70000
INTERSECT
SELECT id FROM employees WHERE department = 'Engineering';

-- EXCEPT: high earners who are NOT in Engineering
SELECT id FROM employees WHERE salary > 70000
EXCEPT
SELECT id FROM employees WHERE department = 'Engineering';

-- DISTINCT vs COUNT(DISTINCT)

```

```
SELECT DISTINCT department FROM employees;
SELECT COUNT(DISTINCT department) FROM employees;
```

Friday — Review + Business Analytics Mini-Project (45 min)

Mini-Project: Create a sales database and write 8 analytical queries.

```
CREATE TABLE products (id SERIAL PRIMARY KEY, name VARCHAR(200), price
NUMERIC(10,2), category VARCHAR(100));
CREATE TABLE orders (id SERIAL PRIMARY KEY, customer_id INTEGER, order_date DATE
DEFAULT CURRENT_DATE);
CREATE TABLE order_items (order_id INTEGER, product_id INTEGER, quantity INTEGER,
unit_price NUMERIC(10,2));

-- Load realistic data (5 products, 3 customers, 10 orders, 20 items)
-- Required queries:
-- 1. Total revenue per product
-- 2. Top 3 customers by total spend
-- 3. Products never ordered (LEFT JOIN + WHERE NULL)
-- 4. Average order value per month
-- 5. Categories with revenue > $1000
-- 6. Most recent order per customer (subquery or window)
-- 7. Revenue comparison: this month vs. last month
-- 8. Full customer + order + item joined view
```

Practice Tasks

1. Write a query that finds departments where the total salary bill exceeds the budget.
2. List products that have been ordered more than 5 times using EXISTS.
3. Combine two queries with UNION to get a unified activity feed.
4. Find employees who earn more than EVERY employee in the HR department (hint: ALL subquery).
5. Show the running total of orders per customer ordered by date.
6. Find the second-highest salary without using LIMIT/OFFSET (use subquery).
7. Self-join: Find pairs of employees in the same department earning within \$10,000 of each other.
8. Write a query that returns department names without any employees using three different approaches (NOT IN, NOT EXISTS, LEFT JOIN + IS NULL).

Self-Assessment Checklist

- ☐ I can explain what LEFT JOIN returns when there's no match
 - ☐ I understand why COUNT(*) differs from COUNT(column_name)
 - ☐ I can write a correlated subquery and explain how it works
 - ☐ I know when to use EXISTS vs IN
 - ☐ I understand the difference between WHERE and HAVING
 - ☐ I can combine result sets with UNION, INTERSECT, EXCEPT
 - ☐ I completed the sales analytics mini-project
 - ☐ I can write 5 queries to analyze sales data without looking at notes
-

Mock Interview Questions

1. What is the difference between INNER JOIN and LEFT JOIN? Give a real-world example.
 2. When would you use NOT EXISTS instead of NOT IN ?
 3. A colleague writes WHERE salary > AVG(salary) . What's wrong with this?
 4. Explain the difference between WHERE and HAVING with an example.
 5. Write SQL to find employees who report to managers earning more than \$100,000.
 6. What does UNION do differently from UNION ALL ? When would you use each?
 7. How do you find duplicate rows in a table?
 8. Explain a self-join with a real-world use case.
 9. What is the performance difference between a subquery in WHERE vs. a JOIN?
 10. Write a query that returns the most recent transaction per user from a transactions table.
-

Resources

- This repo: [03_Intermediate_SQL/](#)
- Visual JOIN reference: <https://www.postgresql.org/docs/16/tutorial-joins.html>
- Practice: <https://pgexercises.com> (joins and aggregates sections)